Sports Tournament Management System

Rupesh Chowdary Peddineni (QuerySquad)
UBIT : rupeshch
UB Person ID: 50665046
Email: rupeshch@buffalo.edu

Vijay kumar Kotta (QuerySquad)
UBIT : vkotta
UB Person ID: 50663289
Email: vkotta@buffalo.edu

***Abstract*—This document represents the overview of Sports Tournament Management System, which uses PostgreSQL database that centralizes players, teams, matches, tournaments, venues, umpires, results, and rankings. It can be applied to any sport, however for instance here we considered cricket. This database overcomes the spreadsheet limits by imposing keys and constraints. This includes Phase-1 and Phase-2 deliverables which defines the problem, target users, E/R model, and the full list of relations with keys and attributes, database creation and testing, transactions and triggers, indexing and query execution analysis.**

I. Introduction

A. Problem Statement

Managing international cricket tournaments involves handling large volumes of dynamic and interrelated data such as player statistics, match results, venues, umpire assignments, team performances, and tournament schedules. So, relying on excel sheets leads to inconsistencies, redundancy, difficulty in analysing data.

This project aims to design a centralized relational database that efficiently stores, retrieves, and manages all tournament-related information. This project is crucial for cricket boards because it reduces operational efficiency generating real-time insights such as performance tracking, live updation of data.

B. What are we going to solve?

This database will answer some key questions such as:

- Which players performed best across different tournaments?
- What are the standings and statistics for each team?
- Which umpires or venues were used most frequently?
- How did each team perform in previous ICC events?
- Who is the winner of a particular tournament?

And many more questions are answered. This will definitely reduce the burden for cricket boards to effectively organise things and manage the tournaments.

C. Why not an Excel file?

While excel file is recommended for small datasets, but in real life scenario we don't work with small datasets. Some of the main reasons includes:

1) Doesn't Support for Relationships Between Entities: Cricket data involve complex relations such as each match involves 2 teams, multiple players, umpires, venues and a tournament can include multiple matches. But, excel stores data in flat sheets – there is no foreign key option here.

For example, if you store matches in one sheet and players in another, if the player's team name changes, we need to manually update it in every sheet which gets easier in case of using relational database.

2) Risk of Human Error: Excel sheet we need to manually edit if we want to make some changes. So there may be chances leading to inconsistent updation of data.

For example, some might store the record as "IND" and some store it as "India" which is incorrect and can be restricted using relational database.

3) Difficult for Real-Time Updates: Matches requires real-time updates and they should be visible instantly. Excel files are not designed to perform such a way. However, the database management system in terms of relational database supports concurrency.

4) Limited Analytical Capability: Excel sheets can't handle complex queries such as Top 10 run-scorers in all tournaments, umpires with highest number of matches contributed etc.

5) No Data Security and Access Control: Excel sheets can accidentallly delete the data but in relational database we can set role-based permissions which limits the access to certain modifications ensuring controlled access.

Above are some of the reasons for choosing relational database over spreadsheets.

D. Target User and Database Administrators

Our database management system is utilised by various people in multiple ways:

- Tournament administrators – manage schedules, tournaments, teams.
- Match officials – record match details and performance data.
- Sport analysts – extract data and analyse it in various means.
- Coaches – analyse team and player performance.

Coming to database administrators, it includes the data operation team in cricket boards such as ICC, BCCI or some technical staff and developers.

E. Real-Life Scenario

Let us consider the upcoming ICC World Cup 2027, this project will allow the system to:

- Get real-time data modifications such as insertions, deletions and updations of match details.
- Query match winners, tournament winners, player stats etc.

- Schedule independent events.

This ensures the ICC, analysts to have unified reliable source of truth for all tournament data.

II. E-R Diagram

Fig.1 represents the E-R diagram for our project. It contains multiple entities with different relationship sets in between as mentioned in above figure.

Below are the brief description of all relationships represented in above E-R diagram

A. Sponsors - Tournament

- Relationship: sponsered
- Meaning: Tracks which sponsers funded which tournament

B. Sponsors – Tournament

- Relationship: sponsored
- Meaning: Tracks which sponsors funded which tournaments.

C. Tournament – Matches

- Relationship: organises
- Meaning: A tournament organizes multiple matches, and each match belongs to one tournament.

D. Matches – Venues

- Relationship: hosted_at
- Meaning: A match is hosted at one venue, and a venue can host multiple matches.

E. Matches – Teams

- Relationship: plays
- Meaning: Two teams play each match, and a team can play multiple matches.

F. Matches – Match Umpires

- Relationship: assigned-to
- Meaning: Multiple umpires can be assigned to a match, and each match umpire role belongs to one match.

G. Umpires – Match Umpires

- Relationship: can-be
- Meaning: An umpire can serve in different match umpire roles.

H. Matches – Scorecards

- Relationship: has-score-report
- Meaning: Each match has one scorecard, and each scorecard belongs to one match.

I. Teams – Players

- Relationship: has=players
- Meaning: A team has multiple players, and a player belongs to one team.

J. Players – PlayerStats

- Relationship: performance
- Meaning: Each player has one performance record (stats), and each performance record belongs to one player.

K. Teams – Points Table

- Relationship: Standings
- Meaning: A team has one standing in the points table, and each points table entry is for one team.

L. Tournament – Points Table

- Relationship: Has Standings
- Meaning: A tournament has multiple entries in the points table, and each points table entry belongs to one tournament.

M. PlayerStats – Matches

- Relationship: has
- Meaning: Each stats record is linked to one match, but a match can have many player stats.

N. Matches - Schedule

- Relationship: scheduled
- Meaning: Each match is scheduled in a venue and has unique record in schedule table.

III. BCNF Normalisation

So, the dataset that I have is mixed version of real-time dataset and program generated version. We as a team, built this database schema, after considering various aspects and making sure the schema is normalised and non-redundant. However, there may be some relation schemas that may be in BCNF or may not be in BCNF. However, we will provide the functional dependencies proving that the relations are already in BCNF or if not, we will try to decompose the relations.

Below are the FD's and BCNF checks on each relation mentioned in ER diagram.

A. Sponsors

Attributes: sponsor_id, sponsor_name, amount, tournament_id

- FD's: sponsor_id $\rightarrow$ sponsor_name, amount; sponsor_id $\rightarrow$tournament_id
- Candidate key: sponsor_id
- BCNF Check: Since LHS is a key, it's in BCNF

B. Tournament

Attributes: tournament_id, tournament_name, format, start_date, end_date, host_country

- FD's: sponsor_id $\rightarrow$ sponsor_name, amount; sponsor_id $\rightarrow$tournament_id
- Candidate key: sponsor_id
- BCNF Check: Since LHS is a key, it's in BCNF

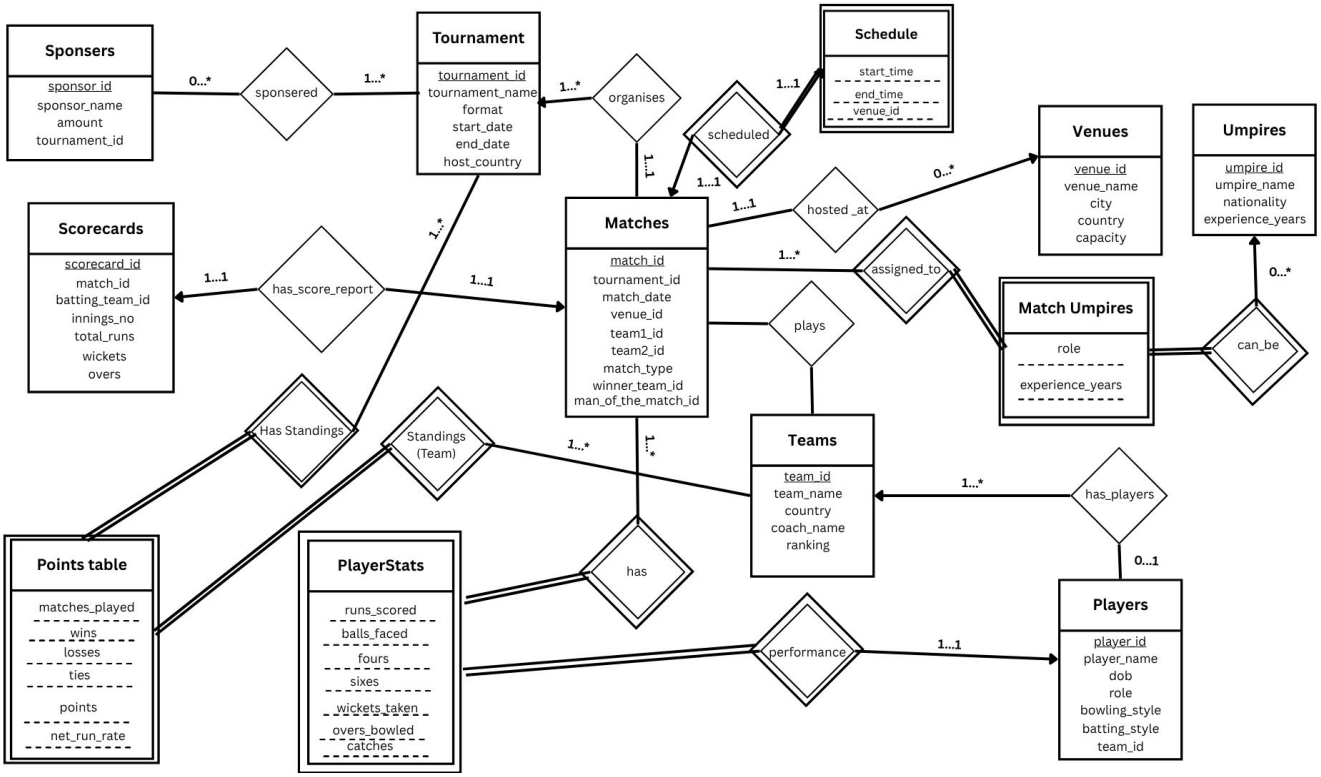


Fig. 1: E-R Diagram

C. Venues

Attributes: venue_id, venue_name, city, country, capacity

- FD's: venue_id → venue_name, city, country, capacity
- Candidate key: venue_id
- BCNF Check: Clearly venue_id is a key, So it's in BCNF

D. Teams

Attributes: team_id, team_name, country, coach_name, ranking

- FD's: team_id → team_name; team_id → country, coach_name; team_id → ranking
- Candidate key: team_id
- BCNF Check: All the FD's have team_id as key which is the determinent. So the relation is in BCNF.

E. Players

Attributes: player_id, player_name, dob, role, bowling_style, batting_style, team_id

- FD's: player_id → player_name,dob; player_id →role, bowling_style, batting_style; player_id → team_id
- Candidate key: player_id
- BCNF Check: Since LHS as the key. So the relation is in BCNF.

F. Umpires

Attributes: umpire_id, umpire_name, nationality, experience_years

- FD's: umpire_id → umpire_name, nationality, experience_years
- Candidate key: umpire_id
- BCNF Check:Since the FD's have umpire_id as key which is the determinent. So the relation is in BCNF.

G. Matches

Attributes: match_id, tournament_id, match_date, venue_id, team1_id, team2_id, winner_team_id, man_of_the_match_id, match_type

- FD's: match_id → tournament_id; match_id → venue_id, team1_id, team2_id; match_id → tournament_id,winner_team_id, man_of_the_match_id, match_type
- Candidate key: match_id
- BCNF Check: Since LHS part is a key. So this relation is in BCNF.

H. Scorecards

Attributes: scorecard_id, match_id, batting_team_id, innings_no, total_runs, wickets, overs

- FD's: (match_id, innings_no) → batting_team_id, total_runs, wickets, overs; scorecard_id→match_id, batting_team_id, innings_no, total_runs, wickets, overs
- BCNF Check: Clearly both FD's do not violate BCNF. Hence, this relation is in BCNF.

I. Schedule

Attributes: match_id, start_time, end_time, venue_id

- FD's: match_id → start_time, end_time; match_id → venue_id
- BCNF Check: Since match_id is the key and is present on LHS of FD it doesn't violate BCNF

J. PlayerStats

Attributes: player_id, match_id, runs_scored, balls_faced, fours, sixes, wickets_taken, overs_bowled, catches

- FD's: player_id, match_id → runs_scored; player_id, match_id → fours, sixes, wickets_taken
- BCNF Check: Clearly player_id, match_id is the key. In every FD, BCNF does not get violated. So the rerlation is in BCNF.

K. PointsTable

Attributes: tournament_id, team_id, matches_played, wins, losses, ties, points, net_run_rate

- FD's: tournament_id, team_id → matches_played; tournament_id, team_id→wins, losses, ties, points, net_run_rate
- BCNF Check: Since, (tournament_id, team_id) is the key and LHS part of every FD is a key. It doesn't violate the BCNF rules.

L. MatchUmpires

Attributes: match_id, tournament_id, match_date, venue_id, team1_id, team2_id, winner_team_id, man_of_the_match_id, match_type

- FD's: match_id, umpire_id → role; umpire_id→ experience_years
- BCNF Check: Clearly umpire_id is not a key So it violates BCNF.

We need to decompose the relation to be in BCNF. So, it can be decomposed such that, the relations look as below

- MatchUmpires(match_id, umpire_id, role)
- UmpireExperience(umpire_id, experience_years)

Since the FD (match_id, umpire_id) → role is in BCNF as match_id, umpire_id are the key. Also, umpire_id → experience_years is also in BCNF since umpire_id became the key after decomposition.

IV. Relations, Attributes, Keys, and Data Descriptions

The following tables are the list of relations representing their respective keys, attributes and the data descriptions. Also, mentioned below what actions are taken on foreign keys when primary key gets deleted.(**This is slightly updated compared to Phase-1**)

In the tables mentioned,

- CASCADE means for example, If a tournament, team, match, or player is deleted, their dependent records (like sponsors, scorecards, etc.) are automatically removed.
- SET NULL is used if an optional reference (like man of the match id) is deleted, the referencing value is set to NULL.
- NO ACTION is used for static data like venue or umpire nationality—deletions are restricted if referenced.

The database consists of 12 relations represented as in below tables. We have represented in tabular format as there are 12 relations and we need to discuss each attribute in each relation which becomes difficult to analyse in text format.

- Each table has a clearly defined primary key ensuring unique identification of records.
- Foreign keys establish relationships such as tournaments organizing matches, matches played by teams, and players belonging to teams.
- Associative entities like MatchUmpires handle many-to-many relations between matches and umpires.
- Referential integrity is maintained using CASCADE or SET NULL rules on foreign keys.
- Attributes are appropriately typed — integers for IDs, decimals for numeric stats, and varchars for textual data.
- Default values (like 0 for stats or points) are set for numeric fields to ensure consistency.
- The schema ensures data normalization, avoiding redundancy while maintaining clear entity relationships.
- Overall, the design supports efficient data retrieval for tournaments, performance tracking, and match management.

V. Database Creation

We have designed and created the database schema with multiple interrelated tables representing tournaments, teams, players, matches, venues, umpires, and related statistics. While defining the relations, we incorporated several data integrity constraints, such as runs ≥ 0, wickets ≤ 10, and others, using the CHECK constraint in PostgreSQL to ensure valid data entry.

Additionally, we carefully defined referential actions for foreign keys, including ON UPDATE cascades to propagate changes automatically and appropriate ON DELETE behaviors (CASCADE, SET NULL, etc.) to maintain consistency across dependent tables.

All the SQL commands and detailed table definitions are documented and available in the create.sql file for reference.

We have populated all the tables with data using SQL commands, resulting in a total of 6,366 records across all relations.

All the code files are available at **https://github.com/kvijaykumar11/Cricket-Tournment-Management-System.git**

VI. Database Testing

We have thoroughly tested our database using over 10 SQL queries, covering a combination of INSERT, UPDATE, DELETE, and SELECT statements. In addition, we implemented stored procedures for certain relations to simplify CRUD operations. Creating procedures for all tables was

TABLE I: Sponsor Table Description

Attribute	Description	Data Type	Key	Null	Default	On Delete
sponsor_id	Unique sponsor identifier	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
sponsor_name	Name of sponsor	VARCHAR(100)		NOT NULL	—	—
amount	Amount contributed	DECIMAL(15,2)		NOT NULL	0.00	—
tournament_id	Tournament being sponsored	INT	**FK → Tournament.tournament_id**	NULL	—	SET NULL

TABLE II: Tournament Table Description

Attribute	Description	Data Type	Key	Null	Default	On Delete
tournament_id	Unique tournament ID	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
tournament_name	Name of tournament	VARCHAR(100)		NOT NULL	—	—
format	Type of tournament (ODI, T20, Test)	VARCHAR(20)		NOT NULL	—	—
start_date	Start date	DATE		NOT NULL	—	—
end_date	End date	DATE		NOT NULL	—	—
host_country	Host country	VARCHAR(50)		NULL	—	—

TABLE III: Teams Table Description

Attribute	Description	Data Type	Key	Null	Default	On Delete
team_id	Team identifier	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
team_name	Name of team	VARCHAR(100)		NOT NULL	—	—
country	Team country	VARCHAR(50)		NOT NULL	—	—
coach_name	Coach name	VARCHAR(100)		NULL	—	—
ranking	Current ICC ranking	INT		NULL	NULL	—

TABLE IV: Players Table Description

Attribute	Description	Data Type	Key	Null	Default	On Delete
player_id	Player identifier	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
player_name	Name of player	VARCHAR(100)		NOT NULL	—	—
dob	Date of Birth	DATE		NULL	NULL	—
role	Role (Batsman, Bowler, All-rounder, etc.)	VARCHAR(50)		NULL	—	—
bowling_style	Bowling type	VARCHAR(50)		NULL	—	—
batting_style	Batting type	VARCHAR(50)		NULL	—	—
team_id	Player's team	INT	**FK → Teams.team_id**	NULL	NULL	SET NULL

TABLE V: Matches Table Description

Attribute	Description	Data Type	Key	Null	Default	On Delete
match_id	Unique match identifier	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
tournament_id	Tournament reference	INT	**FK → Tournament.tournament_id**	NULL	—	SET NULL
team1_id	First team	INT	**FK → Teams.team_id**	NULL	—	SET NULL
team2_id	Second team	INT	**FK → Teams.team_id**	NULL	—	SET NULL
venue_id	Venue reference	INT	**FK → Venues.venue_id**	NULL	—	SET NULL
match_date	Match date	DATE		NOT NULL	—	—
winner_team_id	Winner team	INT	**FK → Teams.team_id**	NULL	NULL	SET NULL
man_of_the_match_id	Player of the match	INT	**FK → Players.player_id**	NULL	NULL	SET NULL
match_type	Type (Group, Final, etc.)	VARCHAR(30)		NULL	—	—

TABLE VI: Venues Table Description

venue_id	Venue identifier	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
venue_name	Name of venue	VARCHAR(100)		NOT NULL	—	—
city	City where venue is located	VARCHAR(50)		NOT NULL	—	—
country	Country	VARCHAR(50)		NOT NULL	—	—
capacity	Stadium capacity	INT		NULL	NULL	—

TABLE VII: Umpires Table Description

umpire_id	Umpire ID	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
umpire_name	Name of umpire	VARCHAR(100)		NOT NULL	—	—
nationality	Country of umpire	VARCHAR(50)		NOT NULL	—	—
experience_years	Matches officiated	INT		NULL	NULL	—

TABLE VIII: PlayerStats Table Description

player_id	Player ID	INT	**FK → Players.player_id**	NOT NULL	—	CASCADE
match_id	Match ID	INT	**FK → Matches.match_id**	NOT NULL	—	CASCADE
runs_scored	Total runs scored	INT		NULL	0	—
balls_faced	Balls faced	INT		NULL	0	—
fours	Number of fours	INT		NULL	0	—
sixes	Number of sixes	INT		NULL	0	—
wickets_taken	Wickets taken	INT		NULL	0	—
overs_bowled	Overs bowled	DECIMAL(4,1)		NULL	0	—
catches	Catches taken	INT		NULL	0	—

TABLE IX: PointsTable Table Description

Attribute	**Description**	**Data Type**	**Key**	**Null**	**Default**	**On Delete**
tournament_id	Tournament reference	INT	**PK, FK → Tournament.tournament_id**	NOT NULL	—	CASCADE
team_id	Team reference	INT	**PK, FK → Teams.team_id**	NOT NULL	—	CASCADE
matches_played	Matches played	INT		NULL	0	—
wins	Matches won	INT		NULL	0	—
losses	Matches lost	INT		NULL	0	—
ties	Matches tied	INT		NULL	0	—
points	Total points	INT		NULL	0	—
net_run_rate	Net run rate	DECIMAL(4,2)		NULL	0.00	—

TABLE X: Scorecard Table Description

scorecard_id	Scorecard ID	INT	**PK**	NOT NULL	AUTO_INCREMENT	—
match_id	Related match	INT	**FK → Matches.match_id**	NOT NULL	—	SET NULL
batting_team_id	Team batting in this innings	INT	**FK → Teams.team_id**	NOT NULL	—	SET NULL
innings_no	Innings number	INT		NOT NULL	—	—
total_runs	Total runs scored	INT		NULL	0	—
wickets	Total wickets	INT		NULL	0	—
overs	Overs faced	DECIMAL(4,1)		NULL	0	—

TABLE XI: Umpire Experience Table Description

umpire_id	Umpire reference	INT	**PK, FK → Umpires.umpire_id**	NOT NULL	—	CASCADE
experience_years	Umpire years of experience	INT		NULL	NULL	—

TABLE XII: Match Umpires Table Description

match_id	Match reference	INT	**PK, FK → Matches.match_id**	NOT NULL	—	CASCADE
umpire_id	Umpire reference	INT	**PK, FK → Umpires.umpire_id**	NOT NULL	—	CASCADE
role	Umpire role	VARCHAR(50)		NULL	NULL	—

challenging, as CRUD operations can vary depending on the specific use case. The results of these tests have been captured and are presented in the images below for your reference (**Fig.2-5**).

Below are the explanations of the queries mentioned from Fig.2-5. To see the SQL files you can download from **https://github.com/kvijaykumar11/Cricket-Tournment-Management-System.git** under Task 5 folder.

A. INSERT, UPDATE, and DELETE Queries

- **Insert and Fetch that new team:** Inserts a new record into Teams, adding name, country, coach, and ranking etc. and selects the team with ranking 53 to confirm the insert succeeded.
- **Insert a new match:** Adds a new match record with tournament, teams, venue, and date and retrieves matches on '2025-02-12' to verify the insert.
- **Fetch updated player:** Changes a player's team_id using UPDATE with a WHERE filter and selects that player to confirm the update took effect.
- **Update match winner and man of the match:** Updates specific fields in a Match using UPDATE and retrieves the match to verify updated winner and man-of-the-match.
- **Delete a sponsor:** Removes a sponsor record using DELETE with a WHERE clause and confirm the deetion using SELECT query.
- **A Venue Deletion:** Deletes a venue record by venue_id using DELETE.and selects that venue to ensure the delete worked.

B. SELECT Queries

- **Total Rows Across All Tables:** Calculates the total number of records by summing the row counts of twelve

tables using multiple subqueries.

- **Player Count by Role:** Groups players by their role using **GROUP BY**, counts them, and sorts the results in descending order using **ORDER BY**.
- **Teams Having More Than 10 Players:** Uses a **JOIN** between Players and Teams, filters grouped results with **HAVING**, and displays the top ten teams using **LIMIT** 10.
- **Highest Scoring Innings in Scorecard:** Selects innings records and sorts them by total runs using **ORDER BY**, returning the highest-scoring ten innings with **LIMIT**.
- **Matches with Tournament and Venue Details:** Combines Matches, Tournament, and Venues using **JOINs**, orders the results by match date, and limits the output to 50 rows.
- **Player Appearances Based on Scorecards:** Uses **LEFT JOINs** and **GROUP BY** to count player appearances, orders them in descending order, and limits the list to 20 players.
- **Highest Points per Team in a Tournament:** Joins PointsTable with Teams, groups records using **GROUP BY**, finds the maximum points using **MAX()**, and sorts the results with a **LIMIT** of 50.

C. Stored Procedures

- **insert_player:** Inserts a new player record into the Players table using the provided attributes.
- **insert_sponsor:** Adds a new sponsor entry with name, amount, and related tournament.
- **update_tournament_dates:** Updates the start and end dates of a tournament using the tournament ID.
- **delete_scorecard:** Deletes a scorecard entry from the Scorecard table using a given ID.

VII. Transaction and Triggers

Its very important to handle the transaction failures using triggers. We considered 2 cases to show transaction failure handling.

First we create a log table to ensure the error logs are being recorded. Here, we created **ScorecardFailures** table to log invalid attempts to insert or update the **Scorecard relation**.

Later we have written a row-level trigger function where it checks whether there is invalid data entry, if so, it logs the error into the ScorecardFailures table and makes sure the insert/update on Scorecard table doesn't happen, aborting the transaction.

First we check with a valid insertion, so trigger fires but no transaction failure as the input data entry is valid. Later, we try with an invalid data entry (inserting 12 wickets ($\geq$10)) where the trigger fires and make sures that insertion won't happen.

For your reference, the screenshots of this scenario are attached below. See **Fig. 6.**

In addition to that, we also implemented another simple transaction failure handling on **PlayerStats Table**. This also works as the same, whenever there is an invalid data (such as setting runs_scored = -10, which is invalid) trying to get updated or inserted into the table, the trigger fires and raises an EXCEPTION, which aborts all the transactions made before that call and logs the errors in to the log table. Also, we are printing the error in the console which can be seen in the image.

You can find the screenshots attached below (**Fig. 7**). However, for detailed observation, you can find the SQL files at **https://github.com/kvijaykumar11/Cricket-Tournment-Management-System.git** under the folder of Task 6.

VIII. Indexing and Query Execution Analysis

We found some problematic queries which can effect the performance and requires improvement. You can find the costs, improved query and cost, indexing used for all three queries below from Fig. 8-12

A. Problematic Query-1

The motive of this query-1 is to get info from the database such that "For each team in Tournament 1, what is the average ranking of their opponents and how many matches did they play?"

The original query is slow because for every **Teams t** row you run a subquery scanning **Matches m** and inside that subquery you run scalar sub-queries. Correlated sub-queries often cause repeated scans or many lookups. Also, the **WHERE** (m.team1_id = t.team_id OR m.team2_id = t.team_id) prevents the planner from using a single efficient index scan on Matches — it often results in two separate index scans or a full table scan depending on stats and selectivity. You can find the query and its cost in **Fig. 8**

The improved query performs significantly faster because it removes correlated subqueries and OR conditions that forced PostgreSQL into repeated rescans, nested loop executions, and thousands of scalar index lookups. By introducing a normalized helper table (MatchTeams), the query now performs simple joins supported by well-targeted indexes.

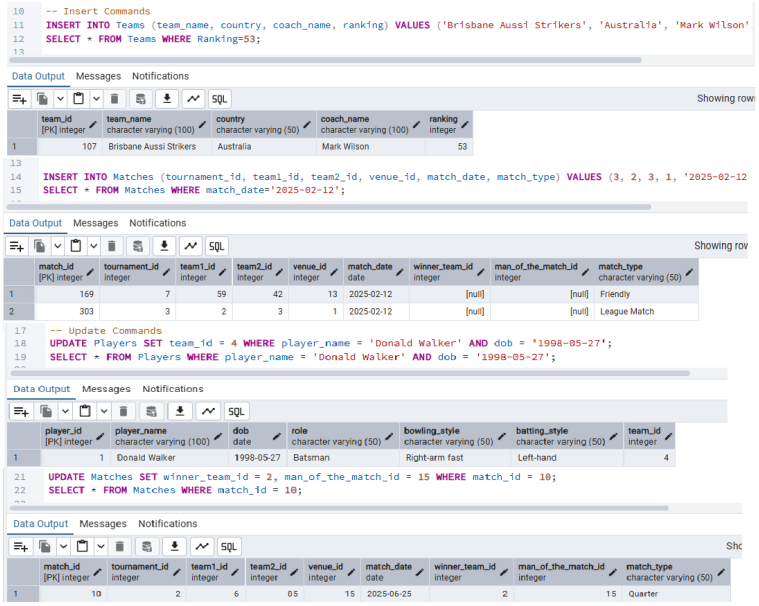
Here, we use **composite index** to get all rows for team_id and tournament_id = 1. This lets the JOIN to be efficient. If we had separate non-composite indexes, the planner might need to intersect indexes or do extra work. Composite indexes can produce index-only scans when the indexed columns cover the query. For detailed view , see **Fig.9**

B. Problematic Query-2

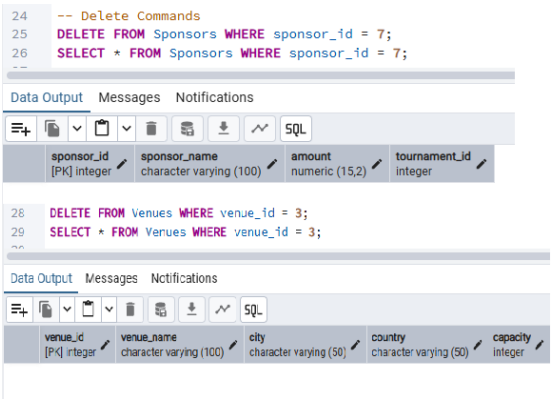
The query is trying to rank umpires based on how many matches they officiated in 2024, but only for umpires who have officiated more than 5 matches overall. You can find in **Fig. 10.**

The original query uses correlated subqueries, which execute once per umpire, **causing 250+ repeated scans** across **Match_Umpires, Matches, and UmpireExperience**. It's leading to multiplying the cost and unnecessary repeated hashing, scanning, and aggregation work, making the query significantly slower.

The improved query aggregates all match counts and experience once in CTEs, eliminating repeated subquery execution.



(a) Insert and Update Operations



(b) Delete Operations

Fig. 2: CRUD Operations Testing (Insert, Update, Delete)

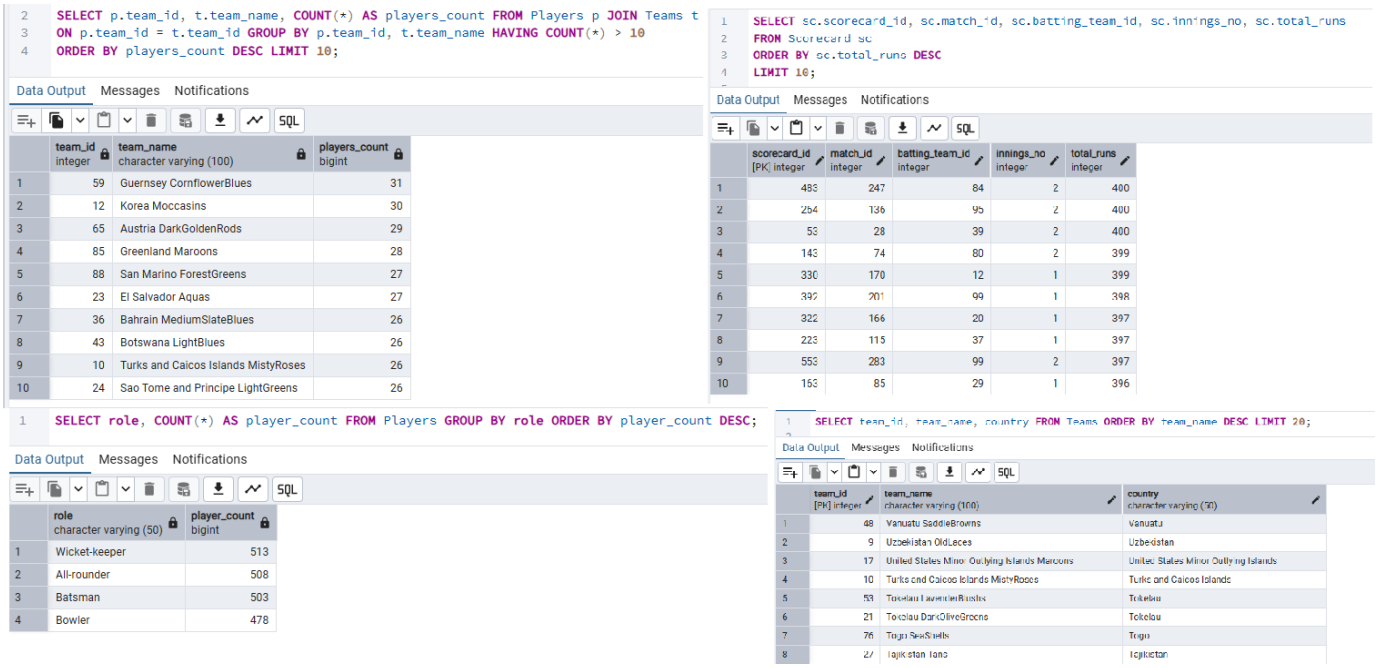


Fig. 3: Retrieval of Records (Select Queries) – Part 1

Hash joins and hash aggregates operate on full tables only a single time, which is extremely efficient.As a result, the final join with Umpires handles already-prepared data, reducing execution time by 3–4×.

The key index that helps both versions is idx_matchumpires_umpire_match (umpire_id), used for fast lookup of match assignments per umpire.In the original query, this index is used repeatedly in bitmap index scans. In the improved query, indexing helps indirectly by keeping table scans and joins cheaper having greater performance improvement.

C. Problematic Query-3

The query is trying to fetch information such that which venues had the highest scoring matches in 2024, considering only those with at least 10 innings?. You can find the query and its cost in **Fig. 11.**

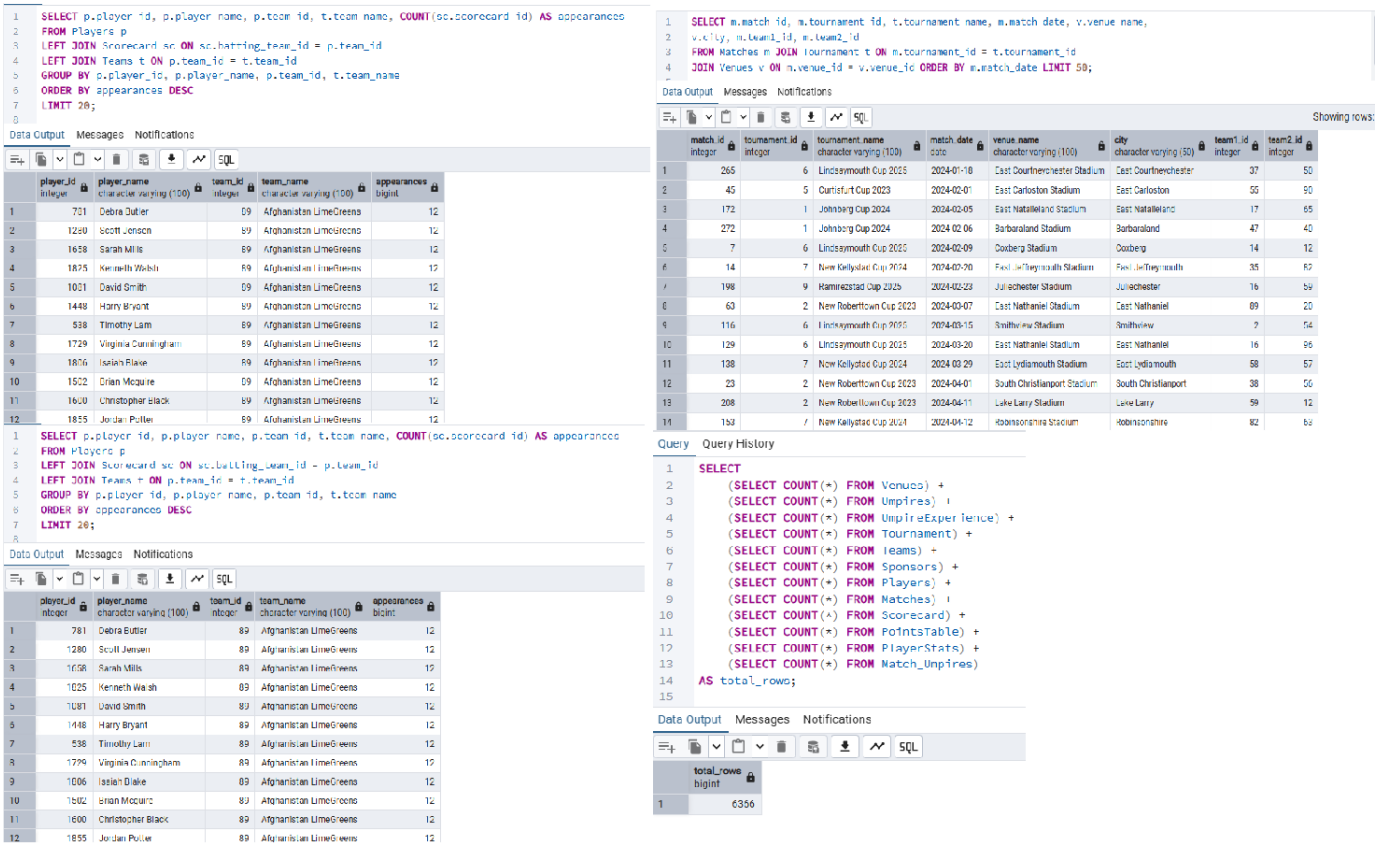


Fig. 4: Retrieval of Records (Select Queries) – Part 2

The original query scans the entire **Scorecard** and **Matches tables** before joining, producing very large intermediate results.The GROUP BY and aggregation happen on all rows, increasing memory and I/O cost.Without selective filtering first, hash joins and aggregation become expensive, especially on a large Scorecard table.

The improved query filters Matches by date first, reducing the number of rows joined to Scorecard. Hash aggregation and sorting operate on a much smaller dataset.This minimizes memory usage, I/O, and execution time while still producing the correct top venues. Also indexing made the search efficient reducing complexity compared to original query.

Coming to indexes, **idx_matches_date** allows fast lookup of matches in 2024 without a full scan.Also, **idx_scorecard_matchid** speeds up the join from Matches to Scorecard. These indexes help the planner reduce scanned rows, enabling efficient hash joins and aggregation.

IX. Conclusion

Our project is based on a real-world scenario. This report provides a comprehensive overview of our database, covering everything from its use cases to its creation, ER models, constraints, sample queries, and examples of transaction failure handling. It also addresses potentially problematic queries and demonstrates how indexing can improve performance, supported by screenshots and visual representations where necessary. This report fulfills all the requirements and provides a complete understanding of our database.

X. Contributions

The following is a tabular representation of our contributions as an individual member. We genuinely present our contributions and we worked hard to complete this project, making sure overall contribution of an individual is equal distributed.

	rupeshch(Rupesh)	vkotta (Vijay)
Task-1	50%	50%
Task-2	55%	45%
Task-3	45%	55%
Task-4	50%	50%
Task-5	50%	50%
Task-6	55%	45%
Task-7	45%	55%

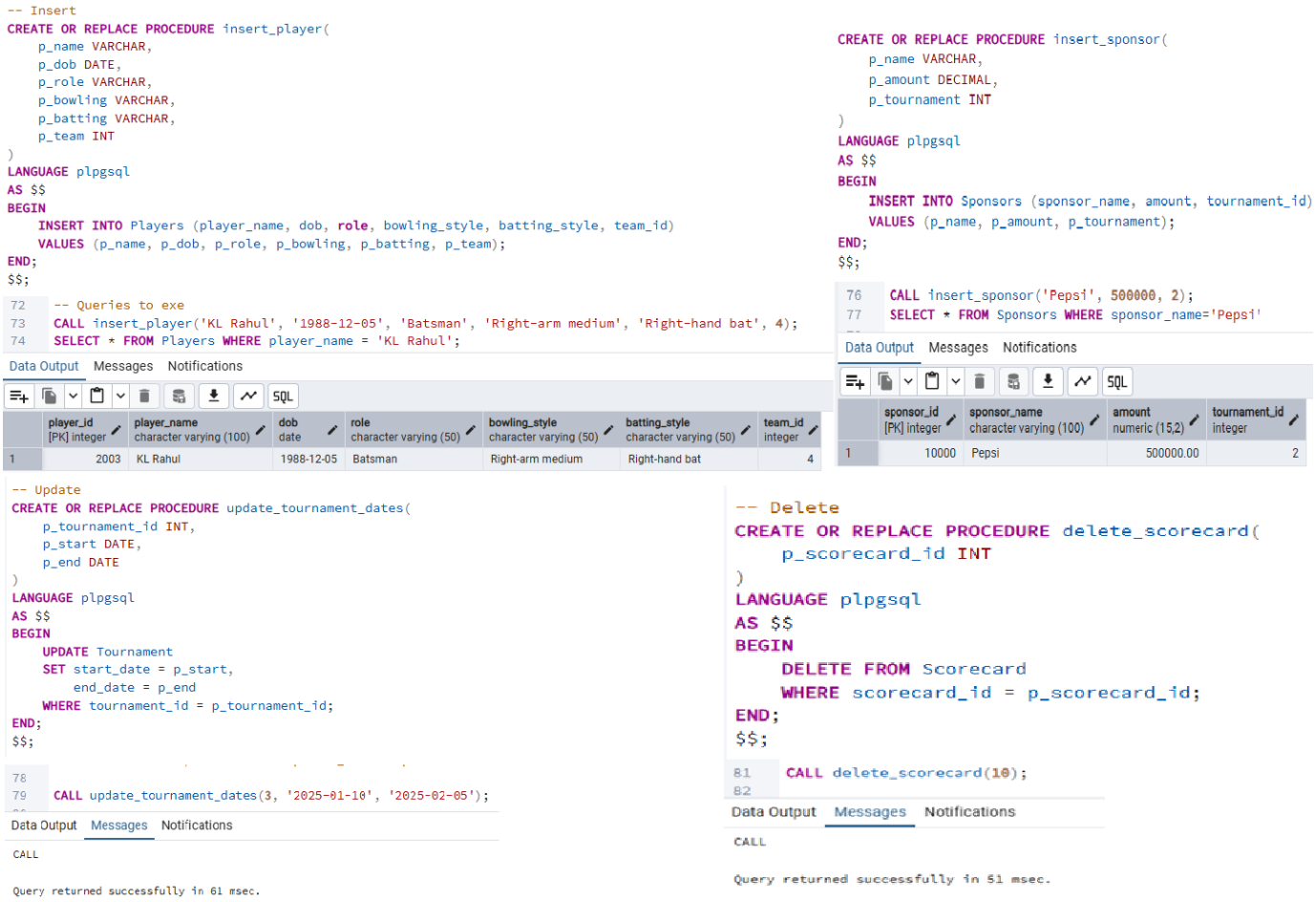


Fig. 5: Some Stored Procedures

```sql
DROP TABLE IF EXISTS ScorecardFailures CASCADE;

CREATE TABLE ScorecardFailures (
    failure_id SERIAL PRIMARY KEY,
    failed_at TIMESTAMP DEFAULT NOW(),
    operation TEXT,
    match_id INT,
    batting_team_id INT,
    innings_no INT,
    error_message TEXT
);

CREATE OR REPLACE FUNCTION check_scorecard_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.total_runs < 0 OR NEW.wickets < 0 OR NEW.wickets > 10 OR NEW.innings_no < 1
       OR (NEW.overs * 10)::INT % 10 > 5 OR NEW.overs < 0 THEN

        INSERT INTO ScorecardFailures(operation, match_id, batting_team_id, innings_no, error_message)
        VALUES (
            TG_OP || ' Scorecard',
            NEW.match_id,
            NEW.batting_team_id,
            NEW.innings_no,
            'Invalid values in Scorecard'
        );

        RETURN NULL;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```sql
-- Creation of Trigger
DROP TRIGGER IF EXISTS trg_check_scorecard ON Scorecard;
CREATE TRIGGER trg_check_scorecard
BEFORE INSERT OR UPDATE ON Scorecard
FOR EACH ROW
EXECUTE FUNCTION check_scorecard_trigger();

-- Valid Transaction Query
INSERT INTO Scorecard(match_id, batting_team_id, innings_no, total_runs, wickets, overs)
VALUES (1, 10, 1, 250, 7, 50.0);

-- Invalid Transaction Query
INSERT INTO Scorecard(match_id, batting_team_id, innings_no, total_runs, wickets, overs)
VALUES (2, 10, 1, 200, 12, 50.3);

-- Query to check logs
SELECT * FROM ScorecardFailures
```

Data Output | Messages | Notifications

	failure_id [PK] integer	failed_at timestamp without time zone	operation text	match_id integer	batting_team_id integer	innings_no integer	error_message text
1	1	2025-11-22 21:49:53.203778	INSERT Scorecard	2	10	1	Invalid values in Scorecard

Fig. 6: Transaction Failure Handling 1

Query | Query History

```sql
DROP TABLE IF EXISTS TriggerFailures CASCADE;
CREATE TABLE TriggerFailures (
    failure_id SERIAL PRIMARY KEY,
    failed_at TIMESTAMP DEFAULT NOW(),
    operation TEXT,
    team_id INT,
    tournament_id INT,
    error_message TEXT
);

CREATE OR REPLACE FUNCTION check_points_trigger()
RETURNS TRIGGER AS $$
BEGIN

    IF NEW.points < 0 THEN

        INSERT INTO TriggerFailures(operation, team_id, tournament_id, error_message)
        VALUES ('INSERT/UPDATE PointsTable', NEW.team_id, NEW.tournament_id, 'Points cannot be negative');

        RAISE EXCEPTION 'Transaction aborted: Points cannot be negative for team_id %, tournament_id %', NE
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER trg_check_points ON PointsTable CASCADE;
CREATE TRIGGER trg_check_points
BEFORE INSERT OR UPDATE ON PointsTable
FOR EACH ROW
EXECUTE FUNCTION check_points_trigger();

-- -- Valid Update
```

Data Output | Messages | Notifications

```
Query returned successfully in 90 msec.
```

```sql
        RAISE EXCEPTION 'Transaction aborted: Points cannot
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER trg_check_points ON PointsTable CASCADE;
CREATE TRIGGER trg_check_points
BEFORE INSERT OR UPDATE ON PointsTable
FOR EACH ROW
EXECUTE FUNCTION check_points_trigger();

-- -- Valid Update
UPDATE PlayerStats
SET runs_scored = 60
WHERE player_id = 1 AND match_id = 39;

-- -- Invalid Update
UPDATE PlayerStats
SET runs_scored = -10
WHERE player_id = 1 AND match_id = 39;

-- -- -- Checking the failure log
```

Data Output | Messages | Notifications

```
ERROR:  Transaction aborted: Negative stats for player_id 1, match_id 39
CONTEXT:  PL/pgSQL function check_playerstats_trigger() line 10 at RAISE

SQL state: P0001
```

Fig. 7: Transaction Failure Handling 2

```
explain analyze
SELECT t.team_id,
       t.team_name,
       (
         SELECT AVG(
           CASE
             WHEN m.team1_id = t.team_id THEN (SELECT ranking FROM Teams WHERE team_id = m.team2_id)
             WHEN m.team2_id = t.team_id THEN (SELECT ranking FROM Teams WHERE team_id = m.team1_id)
             ELSE NULL
           END
         )
         FROM Matches m
         WHERE (m.team1_id = t.team_id OR m.team2_id = t.team_id)
           AND m.tournament_id = 1
       ) AS avg_opponent_ranking,
       COUNT(*) AS matches_played
FROM Teams t
```

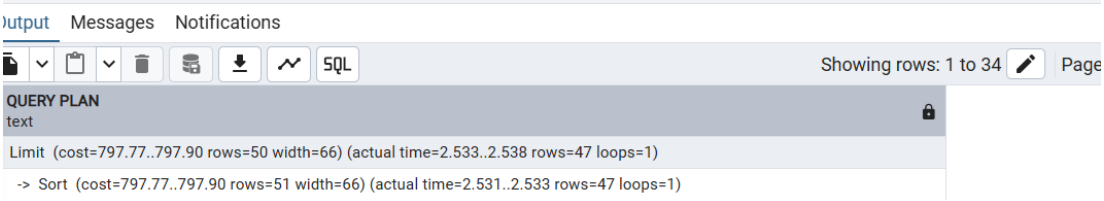


Fig. 8: Performance of Problematic Query 1 (Before Indexing)

```
explain analyze
SELECT t.team_id, t.team_name,
       AVG(op.ranking) AS avg_opponent_ranking,
       COUNT(mt.match_id) AS matches_played
FROM MatchTeams mt
JOIN Teams t ON mt.team_id = t.team_id
JOIN Teams op ON mt.opponent_team_id = op.team_id
WHERE mt.tournament_id = 1
GROUP BY t.team_id, t.team_name
HAVING COUNT(mt.match_id) > 0
ORDER BY avg_opponent_ranking ASC NULLS LAST
LIMIT 50;
```

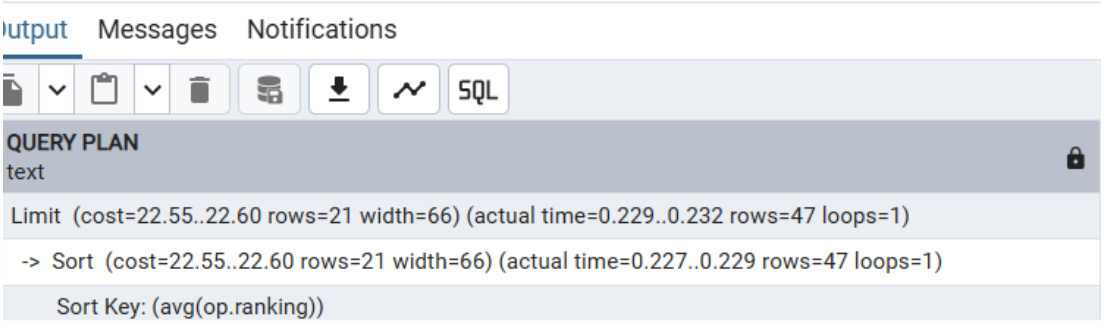


Fig. 9: Performance of Problematic Query 1 (After Indexing)

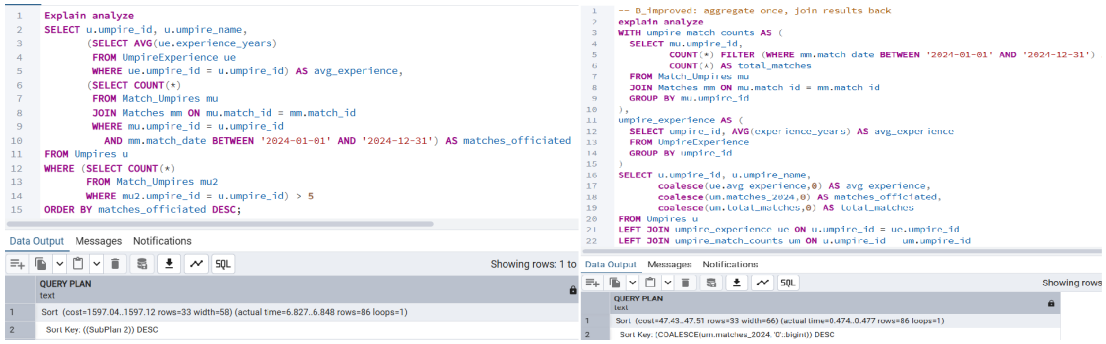


Fig. 10: Performance of Problematic Query 2 (Before and After Indexing)

```sql
EXPLAIN ANALYZE

-- A: top venues by average innings total in 2024
SELECT v.venue_id, v.venue_name, v.city, v.country,
       AVG(sc.total_runs) AS avg_innings_score,
       COUNT(*) AS innings_count
FROM Scorecard sc
JOIN Matches m ON sc.match_id = m.match_id
JOIN Venues v   ON m.venue_id = v.venue_id
WHERE m.match_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY v.venue_id, v.venue_name, v.city, v.country
HAVING COUNT(*) > 10
ORDER BY avg_innings_score DESC
LIMIT 20;

-- Scorecard table is very larger and performing join to the Matches a
--If m.match_date and m.venue_id lack selective indexes, PostgreSql do
```

Data Output | Messages | Notifications

	QUERY PLAN text
1	Limit (cost=40.46..40.51 rows=20 width=498) (actual time=0.915..0.918 rows=0 loops=1)
2	-> Sort (cost=40.46..40.60 rows=53 width=498) (actual time=0.911..0.912 rows=0 loops=1)

Fig. 11: Performance of Problematic Query 3 (Before Indexing)

Data Output | Messages | Notifications

	QUERY PLAN text
1	Limit (cost=40.39..40.44 rows=20 width=498) (actual time=0.897..0.961 rows=0 loops=1)
2	-> Sort (cost=40.39..40.53 rows=53 width=498) (actual time=0.895..0.959 rows=0 loops=1)
3	Sort Key: (avg(sc.total_runs)) DESC
4	Sort Method: quicksort Memory: 25kB
5	-> HashAggregate (cost=36.58..38.98 rows=53 width=498) (actual time=0.883..0.947 rows=0 loops...
6	Group Key: v.venue_id
7	Filter: (count(*) > 10)
8	Batches: 1 Memory Usage: 48kB
9	Rows Removed by Filter: 43
10	-> Hash Join (cost=22.27..35.20 rows=184 width=462) (actual time=0.349..0.793 rows=183 loop...
11	Hash Cond: (m.venue_id = v.venue_id)
12	-> Hash Join (cost=8.68..21.11 rows=184 width=8) (actual time=0.250..0.597 rows=183 loop...
13	Hash Cond: (sc.match_id = m.match_id)
14	-> Seq Scan on scorecard sc (cost=0.00..10.87 rows=587 width=8) (actual time=0.060..0....
15	-> Hash (cost=7.50..7.50 rows=94 width=8) (actual time=0.159..0.159 rows=94 loops=1)
16	Buckets: 1024 Batches: 1 Memory Usage: 12kB
17	-> Seq Scan on matches m (cost=0.00..7.50 rows=94 width=8) (actual time=0.021..0.1...
18	Filter: ((match_date >= '2024-01-01'::date) AND (match_date < '2025-01-01'::date))
19	Rows Removed by Filter: 206
20	-> Hash (cost=11.60..11.60 rows=160 width=458) (actual time=0.079..0.081 rows=50 loops=1)
21	Buckets: 1024 Batches: 1 Memory Usage: 12kB
22	-> Seq Scan on venues v (cost=0.00..11.60 rows=160 width=458) (actual time=0.019..0.0...
23	Planning Time: 1.529 ms
24	Execution Time: 1.187 ms

Fig. 12: Performance of Problematic Query 3 (After Indexing)